

UNIVERSITY INSTITUTE OF COMPUTING

MASTER OF COMPUTER APPLICATIONS

Danish Rashid Pala
Designation: Asst. Professor

UNIT-1

DISCOVER . **LEARN** . EMPOWER

INTRODUCTION To Full Stack Development:

Full Stack Development refers to the development of both front end(client side) and back end(server side) portions of web application.

Technology Related to Full Stack Development

➤ Front-end Development:

It is the visible part of website or web application which is responsible for user experience. The user directly interacts with the frontend portion of the web application or website.

➤ Front-end Technologies:

The frontend portion is built by using some languages which are discussed below:



HTML: HTML stands for Hyper Text Markup Language. It is used to design the front end portion of web pages using markup language. HTML is the combination of Hypertext and Markup language. Hypertext defines the link between the web pages. The markup language is used to define the text documentation within *tag* which defines the structure of web pages.

CSS: Cascading Style Sheets, fondly referred to as CSS, is a simply designed language intended to simplify the process of making web pages presentable. CSS allows you to apply styles to web pages. More importantly, CSS enables you to do this independent of the HTML that makes up *each* web page.

JavaScript: JavaScript is a famous scripting language used to create the magic on the sites to make the site interactive for the user. It is used to enhancing the functionality of a website to running cool *games* and web-based software.

Front End Libraries and Frameworks:

AngularJS:

AngularJS is a JavaScript open source front-end *framework* that is mainly used to develop single page web applications(SPAs). It is a continuously growing and expanding *framework* which provides better ways for developing web applications. It changes the static HTML to dynamic HTML. It is an open source project which can be freely used and changed by anyone. It extends HTML attributes with Directives, and data is bound with HTML.

React.js:

React is a declarative, efficient, and flexible JavaScript library for building user interfaces. ReactJS is an open-source, component-based frontend library responsible only for the view layer of the application. It is maintained by *Facebook*.

Bootstrap:

Bootstrap is a *free* and open-source tool collection for creating responsive websites and web applications. It is the most popular HTML, CSS, and JavaScript *framework* for developing responsive, mobile-first web sites.

jQuery:

jQuery is an open source JavaScript library that simplifies the interactions between an HTML/CSS document, or more precisely the Document Object Model (DOM), and JavaScript. Elaborating the terms, jQuery simplifies HTML document traversing and manipulation, browser event handling, DOM animations, Ajax interactions, and cross-browser JavaScript development.

SASS:

It is the most reliable, mature and robust CSS extension language. It is used to extend the functionality of an existing CSS of a site including everything from variables, inheritance, and nesting with ease.

Some other libraries and frameworks are: Semantic-UI, Foundation, Materialize, Backbone.js, Express.js, Ember.js etc.

Back-end Technologies:

It refers to the server-side development of web application or website with a primary focus on how the website works. It is responsible for managing the database through queries and APIs by client-side commands. This type of website mainly consists of three parts front end, back end, and database.

The Backend portion is built by using some libraries, frameworks, and languages which are discussed below:

PHP: PHP is a server-side scripting language designed specifically for web development. Since, PHP code executed on server side so it is called server side scripting language.

C++: It is a general-purpose programming language and widely used now a days for competitive programming. It is also used as backend language.



Java:

Java is one of the most popular and widely used programming language and platform. It is highly scalable. Java components are easily available.

Python:

Python is a programming language that lets you work quickly and integrate systems more efficiently.

Node.js:

Node.js is an open source and *cross-platform* runtime environment for executing JavaScript code outside of a browser. You *need* to remember that NodeJS is not a *framework* and it's not a programming language. Most of the people are confused and understand it's a *framework* or a programming language. We often use Node.js for building back-end services like APIs like Web App or Mobile App. It's used in *production* by large companies such as PayPal, Uber, Netflix, Walmart and so on.

Popular Stacks:

- ❖ **MEAN Stack:** MongoDB, Express, AngularJS and Node.js.
- ❖ **MERN Stack:** MongoDB, Express, ReactJS and Node.js
- ❖ **Django Stack:** Django, python and MySQL as Database.
- ❖ **Rails or Ruby on Rails:** Uses Ruby, PHP and MySQL.
- ❖ **LAMP Stack:** Linux, Apache, MySQL and PHP.

NodeJS

Node.js was created in 2009 by Ryan Dahl.

Node.js is an open source, *cross*-platform runtime environment and library that is used for running web applications outside the client's browser.

It is used for server-side programming, and primarily deployed for non-blocking, event-driven servers, such as traditional web sites and back-end API services, but was originally designed with real-time, push-based architectures in mind. Every browser has its own version of a JS engine, and node.js is built on Google Chrome's V8 JavaScript engine.

Normally, JavaScript runs in the browser (like Chrome). But **Node.js lets you run JavaScript on your computer/server** — just like you run Python, Java, etc.

◆ Key Features:

Feature	Explanation
JavaScript Everywhere	Use JavaScript for both frontend and backend development
Non-blocking I/O	Handles many requests at once without waiting (asynchronous)
Fast Performance	Built on Google Chrome's V8 engine (very fast at executing JS)
Open-source Modules	Thousands of ready-made packages via npm (Node Package Manager)
Real-time Capability	Great for chat apps, live updates, etc.

Datatypes:

Node.js contains various types of data types similar to JavaScript.

Boolean

Undefined

Null

String

Number

Loose Typing:

Node.js supports loose typing, which means you don't need to specify what type of information will be stored in a variable in advance. We use the var and let keywords in Node.js to declare any type of variable. Examples are given below:

```
// Variable store number data type
```

```
let a = 10;  
console.log(typeof a);
```

```
// Variable store string data type
```

```
a = "NodeJS";  
console.log(typeof a);
```

```
// Variable store Boolean data type
```

```
a = true;  
console.log(typeof a);
```

```
// Variable store undefined (no value) data type
```

```
a = undefined;  
console.log(typeof a);
```

Output:

```
Number  
string  
boolean  
undefined
```



Objects & Functions:

Node.js objects are the same as JavaScript objects i.e. the objects are similar to variables and it contains many values which are written as name: value pairs. Name and value are separated by a colon and every pair is separated by a comma.

Example:

```
let company = {
    Name: "abc",
    Address: "Chandigarh",
    Contact: "+910000000000",
    Email: "abc@cumail.in"
};

// Display the object information
console.log("Information of variable company:", company);

// Display the type of variable
console.log("Type of variable company:", typeof company);
```



Output:

```
Information of variable company: {  
Name: 'abc',  
Address: 'Chandigarh',  
Contact: '+9100000000',  
Email: 'abc@cumail.in' }  
Type of variable company: object
```

Functions:

Node.js functions are defined using the function keyword then the name of the function and parameters which are passed in the function. In Node.js, we don't have to specify datatypes for the parameters and check the number of arguments received. Node.js functions follow every rule which is there while writing JavaScript functions.

Example:

```
function multiply(num1, num2) {
```

```
    // It returns the multiplication of num1 and num2  
    return num1 * num2;}
```

```
// Declare variable
```

```
let x = 2;
```

```
let y = 3;
```

```
// Display the answer returned by
```

```
// multiply function
```

```
console.log("Multiplication of", x, "and", y, "is", multiply(x, y));
```

Node.js REPL (READ, EVAL, PRINT, LOOP)

REPL (READ, EVAL, PRINT, LOOP) is a computer environment similar to Shell (Unix/Linux) and command prompt. Node comes with the REPL environment when it is installed. System interacts with the user through outputs of commands/expressions used. It is useful in writing and debugging the codes.

The work of REPL can be understood from its full form:

Read : It reads the inputs from users and parses it into JavaScript data structure. It is then stored to memory.

Eval : The parsed JavaScript data structure is evaluated for the results.

Print : The result is printed after the evaluation.

Loop : Loops the input command. To come out of NODE REPL, press **ctrl+c** twice



Getting Started with REPL:

To start working with REPL environment of NODE; open up the terminal (in case of UNIX/LINUX) or the Command prompt (in case of Windows) and write node and press 'enter' to start the REPL.

```
C:\Users\Parikshit\Desktop\GeeksForGeeks>node
>
>
>
(To exit, press ^C again or type .exit)
>
```

The REPL has started and is demarcated by the '>' symbol. Various operations can be performed on the REPL. Below are some of the examples to get familiar with the REPL environment.

Example: Performing Arithmetical operations in REPL

```
C:\Users\Parikshit\Desktop\GeeksForGeeks>node
> 10+20
30
> 1-2
-1
> 10/4
2.5
> 10%3
1
>
```

Example: Performing operations using libraries of NODE. **MATH** library is being used in below example.

```
C:\Users\Parikshit\Desktop\GeeksForGeeks>node
> Math.ceil(10/3)
4
> Math.floor(10/3)
3
> math.round(math.e, 3);
ReferenceError: math is not defined
> Math.round(Math.e, 3)
NaN
> Math.round(2.57362, 3)
3
> Math.sqrt(16)
4
> Math.pow(3, 4)
81
>
(To exit, press ^C again or type .exit)
>
C:\Users\Parikshit\Desktop\GeeksForGeeks>node
```

Example: Using variables in REPL. The keyword **var** is used to assign values to variables.

```
C:\Users\Parikshit\Desktop\GeeksForGeeks>node  
> var a = 4;  
undefined  
> a  
4  
> var c = "GeeksForGeeks"  
undefined  
> c  
'GeeksForGeeks'
```

Node.js REPL Commands

Commands	Description
ctrl + c	It is used to terminate the current command.
ctrl + c twice	It terminates the node repl.
ctrl + d	It terminates the node repl.
up/down keys	It is used to see command history and modify previous commands.
tab keys	It specifies the list of current command.
.help	It specifies the list of all commands.
.break	It is used to exit from multi-line expressions.
.clear	It is used to exit from multi-line expressions.
.save filename	It saves current node repl session to a file.
.load filename	It is used to load file content in current node repl session.

What is NPM?

NPM (Node Package Manager) is the **default package manager for Node.js**. It is used to manage libraries, tools, and frameworks (called *packages* or *modules*) that you can use in your Node.js applications.

- It comes automatically install, update installed with Node.js.
- It allows developers to easily , remove, and manage third-party packages.

Key Terms

- **Package:** A reusable block of code (like a library or module).
- **Registry:** A public database where packages are published (<https://www.npmjs.com>).
- **Dependencies:** Packages that your project relies on to function.

Why Use NPM?

- Code reuse:** You don't need to write everything from scratch.
- Version control:** Easily manage versions of libraries.
- Huge ecosystem:** Thousands of open-source packages available.

◆ Basic NPM Commands

Command	Description
<code>npm init</code>	Initializes a new project and creates a <code>package.json</code> file
<code>npm install <package></code>	Installs a package locally in your project
<code>npm install -g <package></code>	Installs a package globally
<code>npm start</code>	Starts the app (if defined in <code>package.json</code>)
<code>npm update</code>	Updates all installed packages
<code>npm uninstall <package></code>	Removes a package

Example

```
npm init -y          # Create a package.json with default settings
npm install express   # Add Express framework
```

```
const express = require('express');
const app = express();

app.get('/', (req, res) => res.send('Hello World!'));

app.listen(3000);
```


NODE JS Modules:

In Node.js, Modules are the blocks of encapsulated code that communicate with an external application on the basis of their related functionality. Modules can be a single file or a collection of multiple files/folders.

The reason programmers are heavily reliant on modules is because of their reusability as well as the ability to break down a complex piece of code into manageable chunks.

Modules are of three types:

- 1. Core Modules**
- 2. local Modules**
- 3. Third-party Modules**





1. Core Modules:

Node.js has many built-in modules that are part of the platform and come with Node.js installation. These modules can be loaded into the program by using the `require` function.

Syntax:

```
const module = require('module_name');
```

The `require()` function will return a JavaScript type depending on what the particular module returns. The following example demonstrates how to use the Node.js `http` module to create a web server.

```
const http = require('http');
http.createServer(function (req, res) {
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.write('Welcome to the world of backend Technologies!');
  res.end();
}).listen(5000);
```

In the above example, the `require()` function returns an object because the `Http` module returns its functionality as an object.

The function `http.createServer()` method will be executed when someone tries to access the computer on port 5000. The `res.writeHead()` method is the status code where 200 means it is OK, while the second argument is an object containing the response headers.

The following list contains some of the important core modules in Node.js:

Core Modules	Description
http	creates an HTTP server in Node.js.
assert	set of assertion functions useful for testing.
fs	used to handle file system.
path	includes methods to deal with file paths.
process	provides information and control about the current Node.js process.
os	provides information about the operating system.
querystring	utility used for parsing and formatting URL query strings.
url	module provides utilities for URL resolution and parsing.

2. Local Modules:

Unlike built-in and external modules, local modules are created locally in your Node.js application. Let's create a simple calculating module that calculates various operations.

Create a eval.js file that has the following code:

Filename: eval.js

```
exports.add = function (x, y) {
  return x + y;
};
exports.sub = function (x, y) {
  return x - y;
};
exports.mult = function (x, y) {
  return x * y;
};
exports.div = function (x, y) {
  return x / y; };
```

Since this file provides attributes to the outer world via exports, another file can use its exported functionality using the `require()` function.

```
const calculator = require('./eval');
```

```
let x = 50, y = 10;
```

```
console.log("Addition of 50 and 10 is "  
    + calculator.add(x, y));
```

```
console.log("Subtraction of 50 and 10 is "  
    + calculator.sub(x, y));
```

```
console.log("Multiplication of 50 and 10 is "  
    + calculator.mult(x, y));
```

```
console.log("Division of 50 and 10 is "  
    + calculator.div(x, y));
```

Step to run this program:

Run the **index.js** file using the following command:
node index.js

Output:

Addition of 50 and 10 is 60

Subtraction of 50 and 10 is 40

Multiplication of 50 and 10 is 500

Division of 50 and 10 is 5

3. Third-party modules:

Third-party modules are modules that are available online using the Node Package Manager(NPM). These modules can be installed in the project folder or globally. Some of the popular third-party modules are Mongoose, express, angular, and React.

Example:

```
npm install express
```

```
npm install mongoose
```

```
npm install -g @angular/cli
```




About Node.js file system:

To handle file operations like creating, reading, deleting, etc., Node.js provides an inbuilt module called FS (File System).

All file system operations can have synchronous and asynchronous forms depending upon user requirements.

To use this File System module, use the require() method:

Var fs = require('fs');

Common use for File System module:

Read Files

Write Files

Append Files

Close Files

Delete Files



Open a File:

The `fs.open()` method is used to create, read, or write a file. The `fs.readFile()` method is only for reading the file and `fs.writeFile()` method is only for writing to the file, whereas `fs.open()` method does several operations on a file. First, we need to load the `fs` class which is a module to access the physical file system.

Syntax:

`fs.open(path, flags, mode, callback)`

Parameters:

Path: It holds the name of the file to read or the entire path if stored at other locations.

Flags: Flags indicate the behavior of the file to be opened. All possible values are (`r`, `r+`, `rs`, `rs+`, `w`, `wx`, `w+`, `wx+`, `a`, `ax`, `a+`, `ax+`).

mode: Sets the mode of file i.e. `r`-read, `w`-write, `r+` -readwrite. It sets to default as readwrite.

err: If any error occurs.

Example:

Let us create a js file named main.js having the following code to open a file input.txt for reading and writing.

```
var fs = require("fs");
// Asynchronous - Opening File

console.log("opening file!");

fs.open('input.txt', 'r+', function(err, fd) {

  if (err) {
    return console.error(err);
  }
  console.log("File open successfully");
});
```

Output:
opening file!
File open successfully



Reading a File:

The `fs.read()` method is used to read the file specified by `fd`. This method reads the entire file into the buffer.

Syntax:

`fs.read(fd, buffer, offset, length, position, callback)`

Parameters:

fd: This is the file descriptor returned by `fs.open()` method.

buffer: This is the buffer that the data will be written to.

offset: This is the offset in the buffer to start writing at.

length: This is an integer specifying the number of bytes to read.

position: This is an integer specifying where to begin reading from in the file. If the position is null, data will be read from the current file position.

callback: It is a callback function that is called after reading of the file. It takes two parameters:

err: If any error occurs.

data: Contents of the file.



Node.js fs.readFile() Method:

The fs.readFile() method is an inbuilt method which is used to read the file. This method read the entire file into buffer. To load the fs module we use require() method. For example: var fs = require('fs');

Syntax:

fs.readFile(filename, encoding, callback_function)

Parameters: The method accept three parameters as mentioned above and described below:

filename: It holds the name of the file to read or the entire path if stored at other location.

encoding: It holds the encoding of file. Its default value is 'utf8'.

callback_function: It is a callback function that is called after reading of file. It takes two parameters:

err: If any error occurred.

data: Contents of the file.

Return Value: It returns the contents/data stored in file or error if any.



Below examples illustrate the fs.readFile() method in Node.js: Example:

```
var fs = require('fs');  
  
// Use fs.readFile() method to read the file  
  
fs.readFile('Demo.txt', 'utf8', function(err, data){  
  
    // Display the file content  
    console.log(data);  
  
});  
  
console.log('readFile called');
```



Path Module:

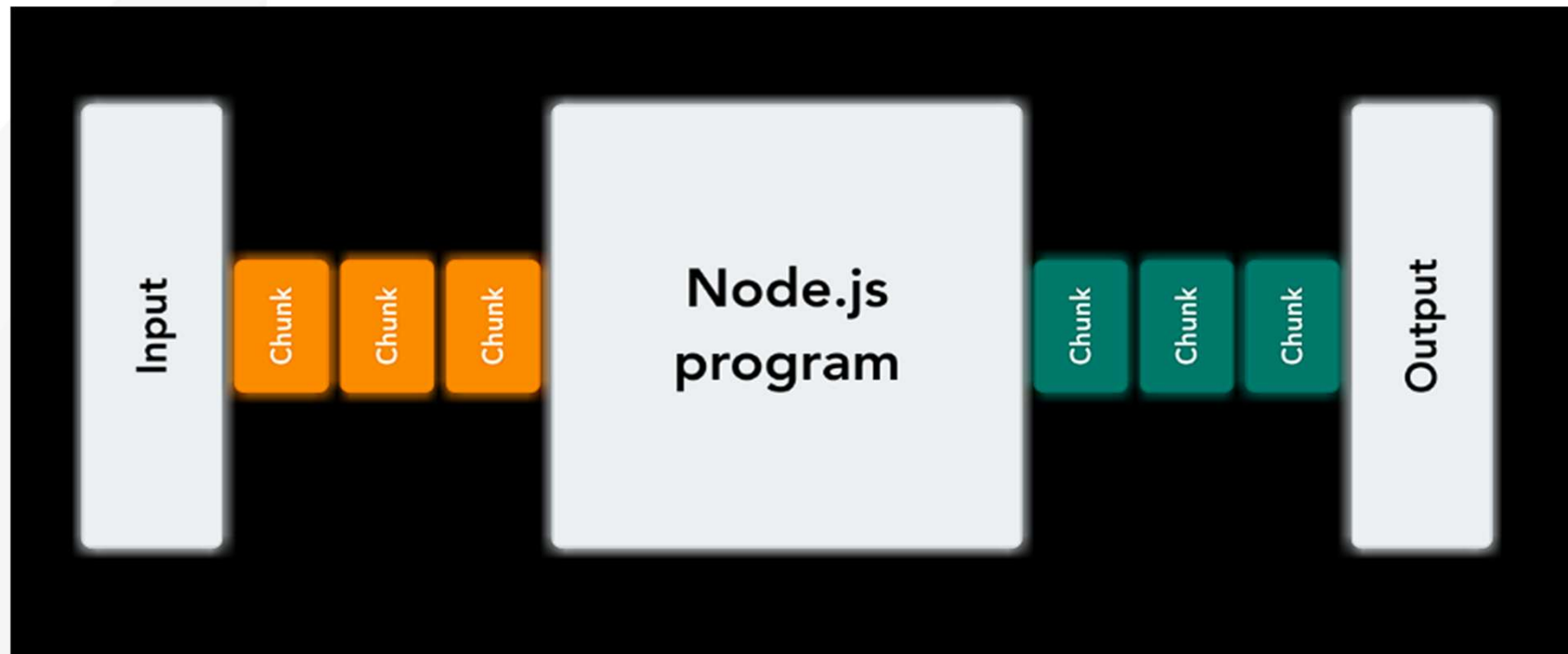
The path module in node.js is used for transforming and handling various file paths.

Syntax:

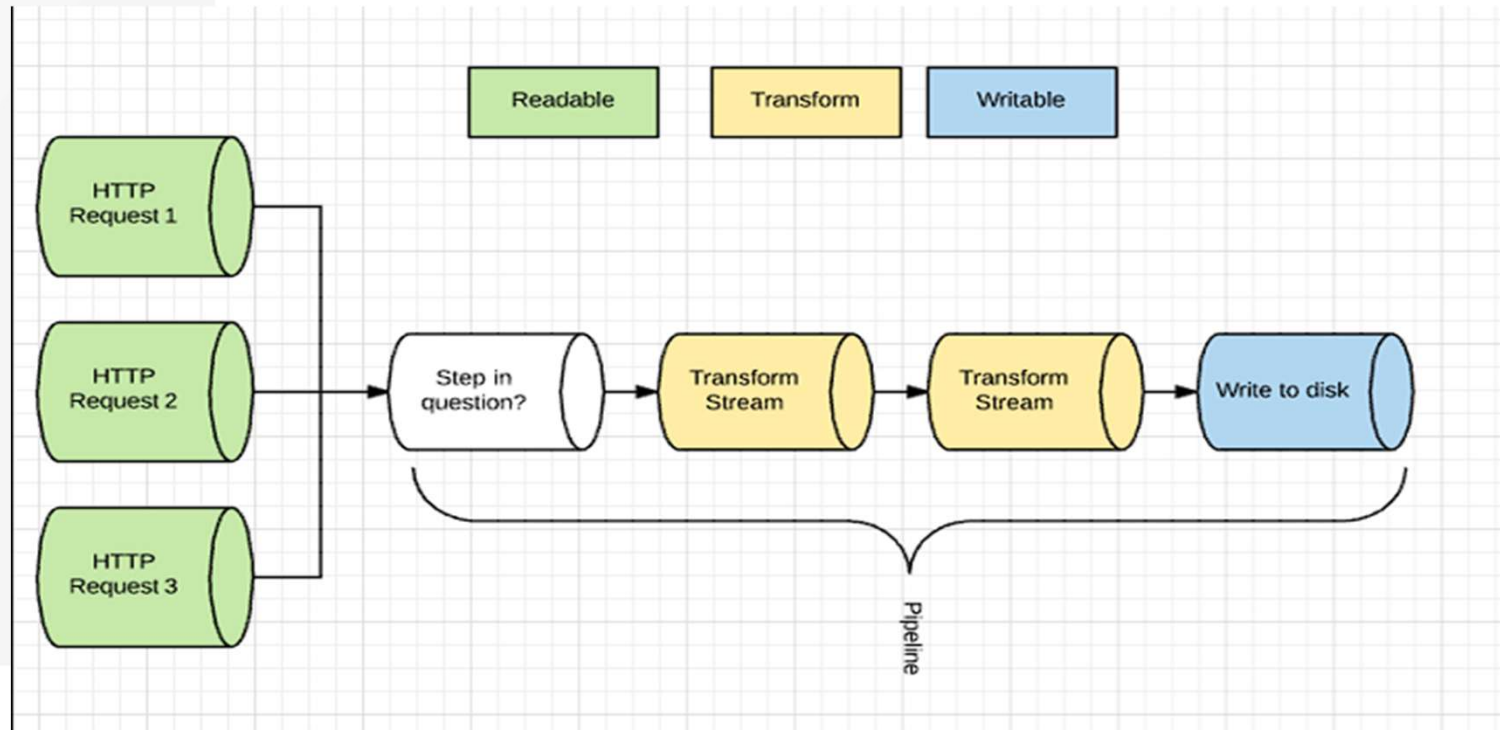
```
const path = require('path');
```

Example:

```
// Require path  
const path = require('path');  
// Display Resolve  
console.log('resolve:' + path.resolve('paths.js'));  
// Display Extension  
console.log('extension:' + path.extname('paths.js'));
```



In Node.js, **streams** are powerful tools for handling large or continuous data flows efficiently. Imagine a conveyor belt moving data piece by piece—this allows your application to process chunks as they arrive, rather than waiting for everything to load into memory first.



Stream and its types in Node.js:

A stream is a way of data handling that helps us to obtain a sequential output by reading or writing the input (files, network communications, and any kind of end-to-end information exchange). That is, they let you read data from a source or write it to a destination or perform any other specific task uninterruptedly and constantly.

The stream is not a unique concept to Node.js and it is a part of Unix for quite a long time. A pipe operator is used to make the programs react with each other by passing streams. Hence, the Node.js stream is used as a basis for all streaming APIs.

Example:

When you are streaming YouTube, Netflix, or Spotify then, instead of the whole content downloading all at once, it downloads in small chunks while you keep browsing.

Another example can be chatting on Facebook or WhatsApp where the data is continuously flowing between two people. This is because instead of reading all the data at once in the memory the stream processes it into smaller pieces to make large files easily readable. It is useful because some files are larger than the available free space that you have on your device. Hence, the stream makes such files readable.

In Node.js, streams are objects that enable efficient reading and writing of data, typically in a continuous, sequential manner. They are a fundamental part of Node.js and are designed to handle large amounts of data by processing it in chunks rather than all at once, which helps with performance and memory efficiency.

When comparing streams to traditional file operations (where you might read or write entire files in one go), streams offer several key benefits. Here's a detailed look at the advantages of using streams over standard file operations:

1. Memory Efficiency

Streams:

- **Chunk-Based Processing:** Streams process data in small, manageable chunks. This means you don't need to load the entire file into memory at once. This is particularly advantageous for handling large files or continuous data sources.
- **Reduced Memory Footprint:** Since only a small portion of the data is kept in memory at any given time, streams significantly reduce the risk of running out of memory, which can be a concern with large files.

2. Performance

Streams:

- **Real-Time Processing:** Streams allow data to be processed and transferred in real-time. For example, you can start processing data as soon as the first chunk is available, without waiting for the entire file to be read.
- **Reduced Latency:** By processing data in chunks, streams can improve application responsiveness and reduce latency, especially in I/O-bound operations.

Traditional File Operations:

- **Delayed Processing:** Traditional file reading or writing often involves waiting for the entire file to be loaded or written before any processing occurs. This can introduce delays and inefficiencies, especially with large files.



3. Scalability

Streams:

- **Scalable Data Handling:** Streams are well-suited for scenarios involving large volumes of data or real-time data streams, such as streaming video, processing logs, or handling large datasets.
- **Pipeline Creation:** Streams can be easily piped together to create data processing pipelines. This modular approach allows for complex processing tasks to be composed from smaller, reusable components.

Traditional File Operations:

- **Less Scalable:** Handling large files with traditional methods can be cumbersome and less scalable, as it may involve loading and processing large amounts of data in a single operation.

5. Error Handling

Streams:

- **Stream-Based Error Handling:** Streams have built-in error handling mechanisms that allow errors to be caught and managed during data processing. The `pipeline` utility simplifies error handling across multiple streams.
- **Graceful Error Management:** Streams provide mechanisms to handle errors gracefully, ensuring that resources are properly cleaned up and that the application can recover or fail gracefully.

Traditional File Operations:

- **Error Handling Complexity:** Error handling with traditional file operations can be more complex, especially when dealing with large files or multiple stages of processing. Errors might need to be managed manually, and cleanup might not be as straightforward.

What Is a Stream?

A **stream** is a way to handle data that arrives over time in chunks—think of a conveyor belt rather than loading everything into RAM at once. This makes processing large files or continuous data (like network requests) efficient in terms of memory and responsiveness.

1. Lower Memory Usage

Streams process data in **small chunks**, not loading the entire file into memory. This makes them ideal for large files—no memory overload.

2. Faster Startup

With streams, processing begins immediately as the first chunk arrives. Unlike `readFile()`, you don't wait for the full file to be read.

3. Built-in Flow Control

Streams handle **backpressure** automatically—if the writer is slow, the reader pauses until it can catch up. This avoids buffering too much data at once.

4. Easy Composition

Streams can be **pipable** together effortlessly.

5. Real-Time Processing

You can process, transform, compress, or encrypt data **while it's being streamed**, with no need for temporary storage.

Types of Streams:

Readable stream:

It is the stream from where you can receive and read the data in an ordered fashion. However, you are not allowed to send anything. For example `fs.createReadStream()` lets us read the contents of a file.

Writable stream:

It is the stream where you can send data in an ordered fashion but you are not allowed to receive it back. For example `fs.createWriteStream()` lets us write data to a file.

Duplex stream:

It is the stream that is both readable and writable. Thus you can send in and receive data together. For example `net.Socket` is a TCP socket.

Transform stream:

It is the stream that is used to modify the data or transform it as it is read. The transform stream is basically a duplex in nature. For example, `zlib.createGzip` stream is used to compress the data using `gzip`.

Core Stream Types

Node.js provides four main stream classes:

- **Readable** – sources you can read from (e.g., file reads, incoming HTTP requests).
- **Writable** – destinations you can write to (e.g., file writes, HTTP responses).
- **Duplex** – streams that are both readable and writable (e.g., network sockets).
- **Transform** – duplex streams that also modify the data passing through (e.g., compression, encryption).

1. Readable Streams

Readable streams are used for reading data. Examples include reading from files, network connections, or HTTP requests.

Key Methods:

- `read(size)`: Reads a specified number of bytes.
- `pipe(destination)`: Pipes the data to another stream (e.g., a writable stream).
- `unpipe(destination)`: Stops piping data to the specified destination.
- `on('data', callback)`: Listens for data events.
- `on('end', callback)`: Listens for the end of the data stream.

2. Writable Streams

Writable streams are used for writing data. Examples include writing to files, network connections, or HTTP responses.

Key Methods:

- `write(chunk, encoding, callback)`: Writes a chunk of data.
- `end(chunk, encoding, callback)`: Ends the stream and optionally writes a final chunk.
- `on('finish', callback)`: Listens for the finish event when all data has been flushed.

3. Duplex Streams

Duplex streams are both readable and writable. Examples include TCP sockets and some HTTP requests.

Key Methods:

- Inherits methods from both readable and writable streams.

Basic Examples

Read a File in Chunks

```
const fs = require('fs');

const readStream = fs.createReadStream('input.txt', { encoding: 'utf8' });

readStream.on('data', chunk => {
  console.log('Received chunk:', chunk);
});

readStream.on('end', () => {
  console.log('Finished reading.');
```

```
});

readStream.on('error', err => {
  console.error('Read error:', err);
});
```

```
const writeStream = fs.createWriteStream('output.txt');
```

```
writeStream.write('First line\n');  
writeStream.write('Second line\n');  
writeStream.end('Last line\n');
```

```
writeStream.on('finish', () => {  
  console.log('Write complete.');
```

```
});  
  
writeStream.on('error', err => {  
  console.error('Write error:', err);  
});
```

You can stream data to a file or any writable destination.

Pipe (Connect) Streams

```
const fs = require('fs');
```

```
const inStream = fs.createReadStream('source.txt');
```

```
const outStream = fs.createWriteStream('dest.txt');
```

```
inStream.pipe(outStream);
```


EXAMPLE:

```
// transform-file.js
const fs = require('node:fs');
const { Transform, pipeline } = require('node:stream');

// 1. Create a Transform stream that uppercases chunk data
const upperCaseTransform = new Transform({
  transform(chunk, encoding, callback) {
    const upper = chunk.toString().toUpperCase();
    callback(null, upper);
  }
});

// 2. Set up file streams
const input = fs.createReadStream('input.txt');
const output = fs.createWriteStream('output.txt');
```

// 3. Pipe input → transform → output, with error handling

```
pipeline( input,  
  uppercaseTransform,  
  output,  
  (err) => {  
    if (err) {  
      console.error('Pipeline failed:', err);  
    } else {  
      console.log('Done! Converted input.txt → output.txt  
(uppercase).');  
    }  
  }  
);
```



What is a Stream Pipeline?

A stream pipeline is a sequence of streams connected together, where the output of one stream becomes the input of the next. The `pipeline` function handles the wiring up of these streams and ensures that data flows from the source to the destination properly.

Key Features of `pipeline`:

1. **Automatic Error Handling:** The `pipeline` function automatically handles errors that occur in any of the streams. If an error is emitted from any stream in the pipeline, it is propagated to the callback function provided to `pipeline`.
2. **End-to-End Data Flow:** `pipeline` connects the streams in a way that data flows from the readable stream, through any intermediate transform streams, to the writable stream.
3. **Cleanup:** When the pipeline completes (either successfully or with an error), it ensures that all streams are properly closed.

What is a callback?

A **callback** is simply a **function you pass into another function**, which gets "called back" later—often after some task or operation finishes .

Why do we use them?

- Node.js is single-threaded—if you waited for each task (like reading a file or waiting for a server response), the whole program would pause.
- Instead, you start the task and hand it a callback. It will notify you (by executing the callback) **once the task completes**, without stopping other work

```
fs.readFile("notes.txt", "utf8", (err, data) => {
  // ← This arrow function is your callback
  if (err) {
    console.error("Error:", err);
    return;
  }
  console.log("File content:", data);
});
```

A callback is simply a function you pass into another function—in this case `fs.readFile`—and it gets called back when the asynchronous task is done.

In your code:
`(err, data) => { ... }`

1. Callback Hell (Pyramid of Doom)

When multiple asynchronous operations depend on each other, callbacks often get deeply nested, forming a pyramid-like structure. This nesting makes code hard to follow and maintain.

2. Poor Readability & Maintainability

Deep nesting leads to code that's messy and difficult to update. Even small tweaks can require reworking numerous nested layers, increasing the risk of bugs.

3. Error Handling Complexity

Each callback often needs its own error-checking logic. Propagating errors across multiple layers is cumbersome and error-prone, and missing one can result in uncaught exceptions.

4. Unpredictable Control Flow

Callbacks are asynchronously executed and may not run in the order that matches the source code. This non-deterministic behavior can cause confusion, especially when sequencing multiple async tasks.

5. Inversion of Control

With callbacks, you pass control of your logic to external functions or third-party APIs. This shifts timing and execution control out of your hands, making understanding and testing harder.



What is Callback Hell:

Callback Hell is when you have multiple nested callbacks, making your code look like a "pyramid of doom" — hard to **read**, **debug**, and **maintain**.

Callback Hell, often referred to as the "Pyramid of Doom," is a situation in JavaScript where multiple nested callback functions are used to perform asynchronous tasks. This leads to deeply indented and hard-to-read code, making it difficult to maintain and debug. It typically occurs when each asynchronous operation depends on the result of the previous one, causing callbacks to be nested within callbacks.

Example of Callback Hell:

```
doTask1(function(err, result1) {
  doTask2(result1, function(err, result2) {
    doTask3(result2, function(err, result3) {
      console.log(result3);
    });
  });
});
```

1. Use Named Functions (Modularization)

Instead of writing everything inside nested callbacks, we define **separate named functions**.

- Code becomes more **readable**
- You can **reuse** functions
- Easier to **test and debug**


```
function task1Callback(err, result1) {
  if (err) return console.error(err);
  doTask2(result1, task2Callback);
}

function task2Callback(err, result2) {
  if (err) return console.error(err);
  doTask3(result2, task3Callback);
}

function task3Callback(err, result3) {
  if (err) return console.error(err);
  console.log("Final Result:", result3);
}

doTask1(task1Callback);
```

This creates a **linear chain** where each step depends on the previous—errors are handled at each level. Compared to anonymous nesting, this approach is more readable and maintainable. Named callbacks avoid creating a "pyramid" of indentation.

2. Use Promises

A **Promise** represents a value that may be available **now**, **later**, or **never**. It helps to run async code in a **thenable** chain instead of deeply nested callbacks.

```
doTask1()
  .then(result1 => doTask2(result1))
  .then(result2 => doTask3(result2))
  .then(result3 => console.log("Final Result:", result3))
  .catch(err => console.error("Error:", err));
```

Remove nested structure
Have better **error handling**
Easy to **chain tasks**



1. Flattened, Linear Structure

Promise chaining replaces nested callbacks with a clean, top-down flow: each `.then()` processes one result and returns the next promise—no more pyramid of doom.

2. Centralized Error Handling

Instead of repeating `if (err)` checks at each layer, you handle errors in one place using `.catch()`.

3. Sequential Task Chaining

Each asynchronous step cleanly follows the previous one. The return value (or promise) from one `.then()` is passed to the next automatically.

4. Readable, Maintainable Code

Promises hide callback nesting and improve readability. They're especially useful when managing dependent tasks or complex workflows.



`async/await` is the **modern and most readable way** to work with asynchronous operations in JavaScript. It is actually built **on top of Promises**, meaning it doesn't replace Promises—it provides a cleaner syntax for using them.

What are `async` and `await`:

- **`async`**: Used before a function declaration to indicate that the function will work with asynchronous operations and automatically returns a Promise.
- **`await`**: Used inside an `async` function to pause execution until a Promise is resolved or rejected.

3. Use Async/Await

async and await are modern JavaScript keywords that make asynchronous code look like synchronous code.

```
async function runTasks() {
  try {
    const result1 = await doTask1();
    const result2 = await doTask2(result1);
    const result3 = await doTask3(result2);
    console.log("Final Result:", result3);
  } catch (err) {
    console.error("Error:", err);
  }
}
```

runTasks();

Looks clean and **easy to read**

Easier to **debug** using breakpoints

Supports **try-catch** for error handling

4. Use Control Flow Libraries (e.g., async)

Node.js has libraries like async, which provide utility functions to handle asynchronous tasks in series, parallel, or waterfall flow.

```
const async = require('async');

async.waterfall([
  function(callback) {
    doTask1(callback);
  },
  function(result1, callback) {
    doTask2(result1, callback);
  },
  function(result2, callback) {
    doTask3(result2, callback);
  }
], function(err, finalResult) {
  if (err) return console.error("Error:", err);
  console.log("Final Result:", finalResult);
});
```

OS Module:

Operating System-based utility modules for node.js are provided by the OS module.

Syntax:

```
const os = require('os');
```

Example:

```
const os = require("os");
```

```
// Display operating System type
```

```
console.log('Operating System type : ' + os.type());
```

```
// Display operating System platform
```

```
console.log('platform : ' + os.platform());
```

```
// Display total memory
```

```
console.log('total memory : ' + os.totalmem() + " bytes.");
```

```
// Display available memory
```

```
console.log('Available memory : ' + os.availmem() + " bytes.");
```

Node.js HTTP Module:

To make HTTP requests in Node.js, there is a built-in module HTTP in Node.js to transfer data over the HTTP. To use the HTTP server in the node, we need to require the HTTP module. The HTTP module creates an HTTP server that listens to server ports and gives a response back to the client.

Syntax:

```
const http = require('http');
```

Example 1:

In this example, we can create an HTTP server with the help of `http.createServer()` method


```
const http = require('http');  
// Create a server  
http.createServer((request, response) => {  
  
  // Sends a chunk of the response body  
  response.write('Hello World!'); //Prints output on browser.  
  
  // Signals the server that all of the response headers and body  
  have been sent  
  response.end();  
  
}).listen(3000); // Server listening on port 3000  
  
console.log("Server started on port 3000"); //Prints output on terminal.
```



Express.js:

Express.js is a minimal and flexible Node.js web application framework.

It is used to build web applications and APIs.

Runs on top of Node.js and simplifies routing, handling requests/responses, middleware, and more.

Think of Express like a foundation or toolkit that helps build websites or web services easily using JavaScript and Node.js.

Why Use Express.js:

- Easy to set up and start a web server.
- Clean and organized **routing** system.
- Built-in support for **middleware** – functions that run during the request-response cycle.
- Helps create RESTful APIs quickly.
- Widely used in the industry — strong community and plugins.



```
const express = require('express');  
const app = express();  
  
// Route for home page  
app.get('/', (req, res) => {  
  res.send('Welcome to Express!');  
});  
  
// Start server  
app.listen(3000, () => {  
  console.log('Server is running on port 3000');  
});
```



Handling Different HTTP Methods

```
app.post('/submit', (req, res) => {  
  res.send('Data submitted');  
});
```

```
app.put('/update', (req, res) => {  
  res.send('Data updated');  
});
```

```
app.delete('/remove', (req, res) => {  
  res.send('Data deleted');  
});
```



What is Routing in Express.js:

Routing refers to how an application handles incoming client requests to a specific URL path and HTTP method (GET, POST, etc.).

Think of routes like instructions for how to handle different types of user requests — like “what to do when someone visits /about or submits a form to /contact”.



Express Route Syntax

`app.METHOD (PATH, HANDLER)`

`app` → The Express application instance.

`METHOD` → HTTP method (like GET, POST, PUT, DELETE).

`PATH` → URL path (e.g., /, /users, /login)

`HANDLER` → Function to handle the request.

Types of HTTP Methods (Focus on GET & POST)

GET

Used to request **data from the server**.

Parameters are passed via the **URL**.

Example: Visiting a webpage or clicking a link.

POST

Used to **send data to the server** (like form submission).

Parameters are passed via the **request body**.

More secure and flexible than GET.





What is body-parser

body-parser is a middleware that parses incoming request bodies before your handlers.

It was originally a separate NPM package (npm install body-parser).

Used to handle form data, JSON, etc.

```
const express = require('express');  
const app = express();
```

```
// Middleware to parse form data  
app.use(express.urlencoded({ extended: true }));
```

```
// Middleware to parse JSON data  
app.use(express.json());
```



The crypto module in Node.js is a built-in module that provides cryptographic functionalities such as:

- Hashing
- Encryption & decryption
- HMAC (keyed hashing)
- Signing and verifying data
- Generating secure random values

It is based on OpenSSL, and is commonly used for securing sensitive data like passwords, tokens, and private communication.

```
const crypto = require('crypto');
```


Multer is a middleware for **Express.js** that helps you **handle file uploads**, especially **multipart/form-data**, which is the encoding used when submitting files through forms.

It processes incoming requests that contain file data and makes the uploaded files accessible through `req.file` or `req.files`

How Multer Works

1. Parses `multipart/form-data`

- This is the encoding used when uploading files.
- Multer reads the file stream and stores it.

2. Saves files to:

- Disk (filesystem) or
- Memory (RAM) or
- You can manually forward it to cloud storage (e.g., AWS S3, Firebase, etc.)

3. Adds file info to request object:

- `req.file` (for single file)
- `req.files` (for multiple files)



Benefits of Multer in Node.js

1. Handles Multipart Form Data Easily

- Multer is specifically designed to handle **multipart/form-data**, the encoding type used for file uploads via HTML forms.
- This makes file handling smooth and reliable.

2. Simplifies File Upload Process

- Multer automatically:
 - Parses the incoming request
 - Extracts file data
 - Saves the files to the specified location (disk or memory)

You don't have to manually parse file data from req.body.

3. Supports Single and Multiple File Uploads

Multer supports:

Single file (.single())

Multiple files (.array())

Uploading from multiple fields (.fields())

This makes it flexible for different form setups.





4. Highly Customizable

- You can configure:
 - **Destination folder**
 - **Filename**
 - **Storage type** (disk or memory)
 - **File size limits**

File type filters (e.g., only allow images)

5. Fast and Lightweight

Multer doesn't use streams or buffers unless needed. It's optimized for performance and small applications.

6. Integrates Well with Express.js

Multer is built as **Express middleware**, making it easy to plug into any Express route.

7. Supports File Validation

You can easily filter files by type, size, or name before accepting them.



8. Temporary In-Memory Storage Option

Useful when you want to process files in memory (e.g., resize images before saving).

```
const storage = multer.memoryStorage();
```

9. Secure by Default

Prevents uploading malicious files by using filters and limits.

You can set limits on:

- File size

- Number of files

- Fields in the form

```
const express = require('express');
const multer = require('multer');
const path = require('path');

// Initialize Express app
const app = express();
const PORT = 3000;

// Configure multer storage
const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    cb(null, 'uploads/'); // directory to save uploaded files
  },
  filename: function (req, file, cb) {
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
    cb(null, uniqueSuffix + path.extname(file.originalname));
  }
});
```

```
// File filter (optional): restrict file types
const fileFilter = (req, file, cb) => {
  // Accept only images
  if (file.mimetype.startsWith('image/')) {
    cb(null, true);
  } else {
    cb(new Error('Only image files are allowed!'), false);
  }
};

// Init upload middleware
const upload = multer({ storage: storage, fileFilter: fileFilter });

// Create uploads directory if it doesn't exist
const fs = require('fs');
if (!fs.existsSync('uploads')) {
  fs.mkdirSync('uploads');
}
```

```

/ Upload endpoint
app.post('/upload', upload.single('myfile'), (req, res) => {
  if (!req.file) {
    return res.status(400).send('No file uploaded.');
```

```

  }
  res.send({
    message: 'File uploaded successfully!',
    filename: req.file.filename,
    path: req.file.path
  });
});
```

// Simple frontend for testing (optional)

```

app.get('/', (req, res) => {
  res.send(`
    <form action="/upload" method="POST" enctype="multipart/form-data">
      <input type="file" name="myfile" />
      <button type="submit">Upload</button>
    </form>
  `);});
```

/ Start the server

```

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```



Component

express

multer

diskStorage

fileFilter

/upload route

Purpose

Create server and routing

Handle file upload logic

Save files to disk with custom names

Restrict uploads to certain types (e.g., images)

API endpoint that accepts and stores the uploaded file



Upload Type	Multer Method	Access Uploaded Files
Single File	<code>upload.single('name')</code>	<code>req.file</code>
Multiple Files (Same Field)	<code>upload.array('name', n)</code>	<code>req.files (array)</code>
Multiple Fields	<code>upload.fields([...])</code>	<code>req.files['fieldname']</code>
Any File (no field check)	<code>upload.any()</code>	<code>req.files (array)</code>

What is Express Generator:

Express Generator is a **command-line tool** provided by the Express.js framework that quickly sets up a **boilerplate (template) Express application**.

Express Generator = a tool that automatically creates the basic structure of an Express application.

Express Generator is a command-line tool that automatically creates the basic folder structure, files, and configuration required to start an Express.js application, saving developers from doing the initial setup manually.

- **Purpose:** It saves time by automatically creating the folder structure, configuration files, and basic code for an Express app so developers can start building features right away instead of setting everything up manually.

Why is it used?

- **Speeds up development:** No need to create folders, files, and basic routes from scratch.
- **Follows standard structure:** Gives a clean, organized project layout that's easy to maintain.
- **Good for beginners:** Shows how an Express project is typically organized.

How to install and use Express Generator:

Step 1: Install

npm install -g express-generator

Step 2: Create a new project

express myapp

express → command

myapp → your project name

Step 3: Go to the project folder

cd myapp



Step 4: Install dependencies

npm install

Step 5: Run the app

npm start

First, we install Express Generator globally. Second, we use the `express` command to automatically create a new Express project. Third, using `cd`, we enter that project folder. After that, we run `npm install` to install the dependencies required by the project, and then we can start our application.

Open <http://localhost:3000> in the browser to see it running.



Basic folder structure:

myapp/

- |
- |— **app.js** # Main application file
- |— **bin/www** # Script to start the server
- |— **package.json** # Project metadata & dependencies
- |
- |— **public/** # Static files (CSS, JS, images)
- |
- |— **routes/** # Routing files (index.js, users.js)
- |
- |— **views/** # Template files (Jade/Pug by default)



When you run **Express Generator**, it doesn't just give you a folder structure; it also **generates ready-to-run code** for a basic Express web app.

You get:

- **app.js** → fully written with route setup, middleware, and error handling.
- **bin/www** → complete server startup script.
- **routes/** → contains working sample routes (index.js and users.js).
- **views/** → includes working sample templates (layout.pug, index.pug, error.pug).
- **public/** → has a sample CSS file so you can see static file serving in action.
- **package.json** → already includes dependencies like express, pug, morgan, etc.



You can literally do:

```
npx express myapp
```

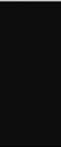
```
cd myapp
```

```
npm install
```

```
npm start
```

It will immediately open a **working Express site** in your browser at <http://localhost:3000>.

From there, you just **edit the generated code** to fit your project idea.





1. app.js — Main Application File

Purpose: This is the **heart** of the Express app.

What it does:

Loads required modules (like express, path, cookie-parser, logger, etc.).

Sets up **middleware** (functions that process requests before they reach routes).

Sets up the **view engine** (like Pug/Jade) to render HTML templates.

Connects routes from the routes/ folder.

Handles errors (404 and other server errors).

In simple terms: It's like the *blueprint* telling Express what to do when a request comes in.





2. bin/www — Server Startup Script

- **Purpose:** This is where the **actual HTTP server** is created and started.
- **What it does:**
 - Imports app.js (the Express app).
 - Decides which port to listen on.
 - Creates an HTTP server using Node's http module.
 - Starts listening for incoming requests.

3. package.json — Project Metadata & Dependencies

Purpose: Keeps all **project information** and **dependencies**.

What it contains:

name, version, description of the project.
 scripts (e.g., "start": "node ./bin/www").
 dependencies (like express, pug, morgan, etc.).
 devDependencies (tools only needed during development).





4. public/ — Static Files Folder

- **Purpose:** Stores files that **don't change dynamically** and are served directly to the client.
- **Typical contents:**
 - stylesheets/ → CSS files
 - javascripts/ → Client-side JS
 - images/ → Image assets

5. routes/ — Routing Files

Purpose: Define how the server responds to different **URLs**.

Default files:

index.js → Handles requests for / (home page).

users.js → Handles requests for /users.

How it works:

Each route file exports a **router object**.

app.js connects them like:



```
var express = require('express');  
var router = express.Router();  
  
/* GET home page. */  
router.get('/', function(req, res, next) {  
  res.render('index', { title: 'Express' });  
});  
  
module.exports = router;
```

This is the home route created automatically by Express Generator.



Benefits of using Express Generator:

1. Rapid Project Setup

- Generates a complete Express app with one command.
- Saves time compared to manually creating folders, files, and configurations.

2. Standard Folder Structure

- Automatically organizes your project into:
 - routes/ → routing files
 - views/ → templates (Pug, EJS, etc.)
 - public/ → static files (CSS, JS, images)
- Makes the app more maintainable and team-friendly.

3. Pre-configured Middleware

- Includes **morgan** (logging), **cookie-parser**, static file serving, and JSON/body parsing.
- You can start coding features without worrying about basic middleware setup.



4. Supports Multiple Template Engines

- Lets you choose Pug, EJS, or Handlebars during setup.
- Saves you from manually integrating view engines.

5. Great for Prototyping

- Perfect for quickly testing ideas or making a proof-of-concept.
- You can extend it into a production app later.

6. Learning-Friendly

- Beginners get a working Express app to study and modify.
- Shows how to organize routes, views, and middleware properly.

Assignment: Build a Basic “Student Info App” using Express Generator with Pug

Objective:

Learn to set up an Express application using **Express Generator**, use **Pug templates**, create routes, and pass dynamic data from server to view.